

TEMA I: INTRODUCCIÓN

→ Los lenguajes más utilizados en la computación científica (especialmente en la Física) son FORTRAN y C++ dentro de los compiladores, y Python dentro de los lenguajes interpretados.

→ Todavía hoy en día hay una gran cantidad de programas disponibles en las bibliotecas científicas escritos en FORTRAN.

→ Sin embargo, en los últimos años, tanto C++ como Python se han hecho cada vez más importantes en la Computación científica.

→ En este curso utilizaremos el lenguaje C++.

→ El compilador de C++ está disponible sin coste alguno tanto para Linux como para Windows:

<https://sourceforge.net/projects/orwelldevcpp/files/Setup%20Releases/>

Algoritmos y programas.

→ Un ordenador tiene una gran capacidad de cálculo pero muy poca “iniciativa”. Para resolver un problema hay que decirle paso a paso lo que tiene que hacer.

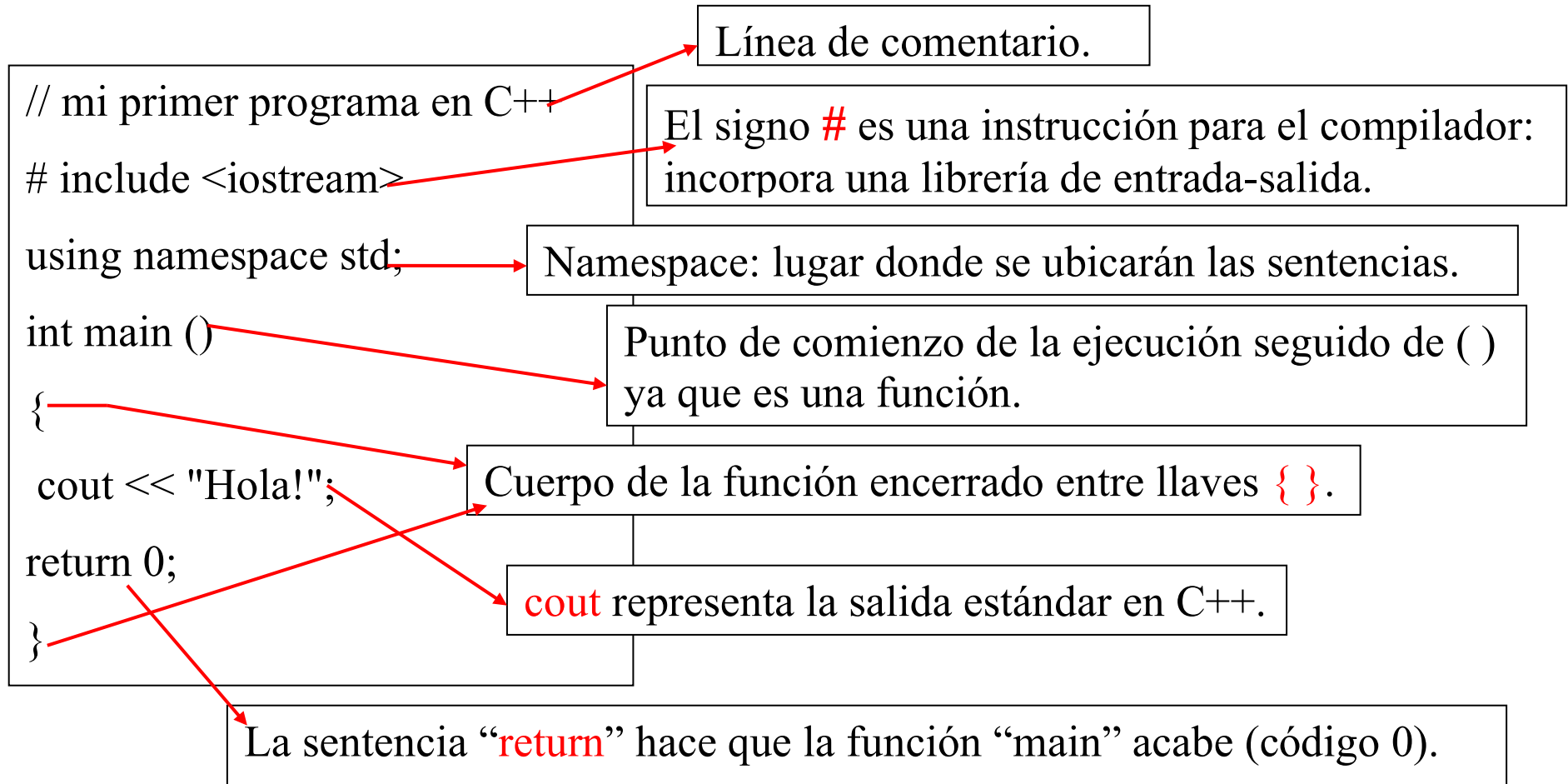
⇒ **Algoritmo**: procedimiento que describe, sin ambigüedades, una serie finita de pasos a realizar en un orden específico para resolver un problema.

→ Los métodos numéricos siempre pueden llegar a una solución numérica, que puede tener una cierta imprecisión, aunque el problema no tenga solución analítica.

⇒ **Programa**: conjunto de instrucciones **completas** y **detalladas** que se le da a un ordenador para que complete una tarea. Siguen el plan dictado por un algoritmo, donde figuran las instrucciones genéricas, sin detallar. Están escritos en un lenguaje de programación (código fuente).

→ La diferencia fundamental entre algoritmo y programa es que el primero no tiene ninguna restricción en su sintaxis, mientras que el segundo necesita un lenguaje concreto y no puede incumplir sus reglas sintácticas.

Estructura de un programa en C++



→ Todas las variables, funciones, tipos, etc..., de la “biblioteca estándar” de C++ pertenecen al espacio de nombres *std*.

→ La línea “using namespace std;” indica que vamos a utilizar dicho espacio de nombres, por lo que no es necesario incluir “**std::**” antes del nombre de cada recurso.

→ El programa anterior ha sido estructurado en diferentes líneas con objeto de ser más comprensible, pero en C++ no existen reglas estrictas sobre como separar instrucciones en diferentes líneas. Por ejemplo, en lugar de:

```
int main ()  
{  
    cout << " Hola!";  
    return 0; }
```

podríamos haber escrito:

```
int main () { cout << "Hola!"; return 0; }
```

→ Todo en una sola línea y tendría el mismo significado que el código previo.

INTRODUCCIÓN AL C++

- 1.- Variables. Tipos de datos.
- 2.- Operaciones básicas de Entrada/Salida: operadores cout y cin.
- 3.- Operadores.
- 4.- Sentencias de control:
 - (a) Ejecución selectiva. Sentencias condicionales: if; if-else.
 - (b) Ejecución repetitiva. Bucles: while; do while; for.
- 5.- Entrada-salida básica con ficheros.
- 6.- Arrays. Punteros.
- 7.- Funciones. Parámetros de una función. Argumentos pasados por valor y por referencia.
- 8.- Reserva dinámica de memoria.

1.- Variables. Tipos de datos.

→ El nombre de una variable es un **identificador** que puede incluir las letras **a** a **z**, **A** a **Z**, los dígitos **0** a **9** y el guión bajo, "**_**". No puede empezar por un dígito.

→ Ejemplos:

```
lambda=0.300;
```

```
limite_superior=2.0;
```

El C++ es un lenguaje que distingue entre minúsculas y mayúsculas.

OJO con las palabras reservadas por el compilador!!!

asm	delete	if	return	try
auto	do	inline	short	typedef
bool	double	int	signed	union
break	else	longmutable	sizeof	unsigned
case	enum	namespace	static	virtual
catch	explicit	new	struct	void
char	extern	operator	switch	volatile
class	float	private	template	wchar_t
const	for	protected	this	while
continue	friend	public	throw	
default	goto	register		

Tipos de datos básicos en C++.

Tipo	Descripción	Tamaño	Rango
char	Tipo carácter	1 byte	
int	Enteros (con signo o sin signo)	4 bytes	-2147483648 a 2147483647
float	Números reales en simple precisión	4 bytes	+/- 3.4 e ^{+/- 38}
double	Números reales en doble precisión	8 bytes	+/- 1.7 e ^{+/- 308}

→ Números enteros : **int**. Su representación es exacta.

→ Por el contrario, los números reales en notación de coma flotante no reflejan una cantidad exacta, sino una aproximación a dicha cantidad.

→ **Números reales en simple precisión: float.**

La precisión decimal correspondiente es de 7 cifras decimales.

→ **Números reales en doble precisión: double.**

La precisión decimal correspondiente es de 16 cifras decimales.

→ Todas las variables que queramos usar en el programa **deben haber sido declaradas con su tipo correspondiente antes de su utilización.**

Lo habitual es declararlas al principio del programa.

```
// Ejemplo:
#include <iostream>
using namespace std;
int main ()
{
    // declaramos variables:
    int a, b;
    int result;
    a = 5;
    b = 2;
    a = a + 1;
    result = a - b;
    // mostramos el resultado por pantalla: 4
    cout << result;
    // terminamos el programa
    return 0;
}
```

→ Una variable puede ser **global** o **local**. Una variable global es aquella que se declara en el cuerpo principal del programa fuente, fuera de las funciones, mientras que una variable local es la que se declara en el cuerpo de una función o en un bloque.

```
//Ejemplo:  
#include <iostream>  
using namespace std;
```

```
int Numero_entero;  
char nombre;  
char apellido [20];
```

Variables globales.

```
int main()  
{
```

```
    int edad;  
    float A;
```

Variables locales

```
    cout << "Teclea tu edad:";  
    cin >> edad;  
    ....  
}
```

→ **Constantes definidas por el usuario: #define**

Se puede usar un nombre elegido por nosotros para referirnos a una constante que se vaya a usar frecuentemente, simplemente usando la instrucción **#define**.

```
#include <iostream>
using namespace std;
#define PI 3.14159
int main ()
{
    double r=5.0; //radio
    double longitud;
    longitud = 2 * PI * r;
    cout << longitud << endl;
    return 0;
}
```

Las líneas con #include y
#define no acaban en ;

RESULTADO: 31.4159

→ **Constantes declaradas (const).**

Con el prefijo const podemos declarar constantes con un tipo específico de la misma forma que hacemos con una variable:

```
const int ancho = 100;
```

```
const double PI = 3.1415927;
```

```
const char tabulador = "\t";
```

Aquí, “ancho”, “PI” y “tabulador” son constantes de diferentes tipos. Son tratadas como variables normales, pero sus valores no pueden ser modificados después de su definición.

→ **Cadenas de caracteres.**

→ El tipo *char* permite almacenar variables de tipo carácter. Cuando queremos guardar una cadena de caracteres se define con "char*" ó char[]:

```
char *cadena_ejemplo = "hola";
```

→ También se puede utilizar la clase "string", aunque para ello hay que incluir la directiva **"#include <string>".**

```
#include <iostream>
```

```
#include <string>
```

```
string cadena;
```

```
cadena = "adios";
```

2.- Operadores de Entrada/Salida.

cout : operador de salida por pantalla (cout << <expresión ó variable>;).

→ El dispositivo estándar de salida es la pantalla. Para ello se utiliza el operador de inserción (<<) seguido de la expresión o variable que se quiere mostrar en pantalla.

```
cout << "Buenos días"; // imprime "Buenos días" en la pantalla
```

```
cout << 100; // imprime el número 100 en la pantalla
```

```
cout << x; // imprime el valor de x en la pantalla
```

→ Con objeto de introducir un salto de línea debemos insertar explícitamente un carácter específico:

```
cout << "Primera sentencia.\n";
```

```
cout << "Segunda sentencia.\n";
```

→ Una forma alternativa consiste en usar el manipulador de flujo **endl**:

```
cout << "Primera sentencia." << endl;
```

```
cout << "Segunda sentencia." << endl;
```

cin : operador de entrada por teclado (cin >> <valor>;).

→ El dispositivo estándar de entrada es el teclado. Para ello aplicaremos el operador de extracción (>>) seguido de la variable que almacena el dato que quiere ser extraído del teclado. Por ejemplo:

```
#include <iostream>
using namespace std;
int main()
{
    int i;
    cout << "Introduce un valor entero:";
    cin >> i;
    return 0;
}
```

→ Hay que utilizar "using namespace std;" para ubicar los operadores cin y cout en el espacio de nombres estándar. Si no tendríamos que escribir: std::cin y std::cout.

3.- Operadores.

→ Operadores aritméticos:

Las cinco operaciones aritméticas implementadas en C++ son:

+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo

→ **División entera frente a división real:**

`int a = 1/2; // a toma el valor 0, porque a es entero`

`float b = 1/2; // b toma el valor 0.5, porque b es real`

→ **Módulo** es la operación que da el resto de una división de dos valores. Por ejemplo, si escribimos:

`a = 11 % 3;`

la variable a contendrá el valor 2, que es el resto de dividir 11 entre 3.

→ Para realizar operaciones matemáticas más complejas se debe incluir la biblioteca matemática **cmath** mediante la directiva **"#include <cmath>".**

```
// Ejemplo cmath
#include <iostream>
using namespace std;
#include <cmath>

int main () {
double a=5.0, b=2.0;

cout << "El seno de " << a << " es " << sin(a) << endl;
cout << "La exponencial de " << b << " es " << exp(b) << endl;
cout << "El logaritmo neperiano de " << a << " es " << log(a) << endl;
cout << a << " elevado a " << b << " es " << pow(a,b) << endl;
cout << "La raíz cuadrada de " << b << " es " << sqrt(b) << endl;
cout << "El valor absoluto de " << a << " es " << fabs(a) << endl;
return 0;
}
```

`sin(x)` : devuelve el seno de x.

`cos(x)` : devuelve el coseno de x.

`tan(x)` : devuelve la tangente de x.

`asin(x)` : devuelve el arco seno de x. El resultado se devuelve en radianes.

`acos(x)` : devuelve el arco coseno de x. El resultado se devuelve en radianes.

`atan(x)` : devuelve el arco tangente de x. El resultado se devuelve en radianes.

`sinh(x)` : devuelve el seno hiperbólico de x.

`cosh(x)` : devuelve el coseno hiperbólico de x.

`tanh(x)` : devuelve la tangente hiperbólica de x.

`exp(x)` : devuelve la exponencial de x.

`log(x)` : devuelve el logaritmo neperiano de x.

`log10(x)` : devuelve el logaritmo decimal de x.

`pow(x,y)` : devuelve x^y .

`sqrt(x)` : devuelve la raíz cuadrada de x.

`ceil(x)` : devuelve el número entero mayor que o igual a x.

`floor(x)` : redondea por defecto el valor de x.

`fabs(x)` : devuelve el valor absoluto de un número en coma flotante.

`labs(x)` : devuelve el valor absoluto de un entero largo (long).

`abs(x)` : valor absoluto de un número entero.

`fmod(x,y)` : calcula el resto de la división real x/y .

→ **Operador asignación (=)**.

→ Permite dar un valor a una variable o a una constante.

```
// operador de asignación
#include <iostream>
using namespace std;
int main ()
{
    int a, b; // a=?, b=?
    a = 10; // a=10, b=?
    b = 4; // a=10, b=4
    a = b; // a=4, b=4
    b = 7; // a=4, b=7
    cout << "a=" << a;
    cout << " b=" << b;
    return 0;
}
```

Resultados:

a=4 b=7

→ **Operadores de asignación.**

expresión	es equivalente a
$a += b;$	$a = a + b;$
$a -= 5;$	$a = a - 5;$
$a /= b;$	$a = a / b;$
$a *= b;$	$a = a * b;$

Operador Incremento y decremento (++ , --)

→ Los operadores incremento (**++**) y decremento (**--**) incrementan o reducen en uno el valor almacenado en una variable. Son equivalentes a $+=1$ y $-=1$, respectivamente.

Por tanto:

$a++;$

$a+=1;$

$a=a+1;$

son todas ellas formas equivalentes: las tres incrementan en uno el valor de a.

Ojo:

$x=y++$; (posincremento)

el resultado es: $x=y$; $y=y+1$;

en cambio, si:

$x=++y$; (preincremento)

el resultado es: $y=y+1$; $x=y$;

Operador	Llamado	Expresión de ejemplo	Explicación
++	preincremento	++a	Incrementa a en 1, posteriormente utiliza el nuevo valor de a en la expresión en donde reside.
++	posincremento	a++	Utiliza el valor actual de a en la expresión en la que reside, y posteriormente incrementa a en 1.
--	predecremento	--b	Decrementa b en 1, posteriormente utiliza el valor de b en la expresión en la que reside.
--	posdecremento	b--	Utiliza el valor actual de b en la expresión en la que reside, posteriormente disminuye b en 1.

→ **Operadores relacionales:**

→ Sirven para comparar dos operandos. El resultado de la operación de comparación es un valor que puede ser verdadero (cualquier valor distinto de 0) o falso (0).

Operador	Descripción	Ejemplos
<	Menor que	(5 < 5) // falso
<=	Menor o igual que	(5 <= 5) // verdadero
>	Mayor que	(5 > 4) // verdadero
>=	Mayor o igual que	(6 >= 6) // verdadero
==	Igual a	(7 == 5) // falso
!=	Distinto de	(3 != 2) // verdadero

→ Si definimos a=2, b=3 y c=6;

(a == 5) // falso.

(a*b >= c) // verdadero ya que (2*3 >= 6) es verdadero.

(b+4 > a*c) // falso ya que (3+4 > 2*6) es falso.

(b=2) == a) // verdadero

→ Constituyen, junto con los **operadores lógicos**, los bloques de construcción básicos para la toma de decisiones.

Negación lógica	!a
AND lógico	a && b
OR lógico	a b

4. Sentencias de control. Ejecución selectiva.

→ Estructura condicional if:

La estructura if se usa para ejecutar una sentencia o un bloque de sentencias solamente si se cumple una determinada condición. Por ejemplo:

```
if (x == 100)  Expresión lógica  
    cout << "x vale 100";
```

sacará por pantalla “x vale 100” solamente si el valor almacenado en la variable x es 100.

→ Si quisiéramos que se ejecutase mas de una sentencia en el caso de que se cumpla la condición, podemos especificar un bloque mediante el uso de llaves { }:

```
if (x == 100)  
{  
    cout << "x vale ";  
    cout << x;  
}
```

→ Estructura condicional: if-else

→ Podemos especificar adicionalmente lo que queremos que ocurra si no se cumple la condición usando la instrucción if-else.

Por ejemplo:

```
if (x > 0)
    cout << "x es positiva" << endl;
else if (x < 0)
    cout << "x es negativa" << endl;
else
    cout << "x es 0" << endl;
```

→ Podemos utilizar tantas líneas else-if como sean necesarias.

Estructuras iterativas (bucles). Ejecución repetitiva.

(a) - Instrucción **while** : se repite una sentencia o bloque de sentencias mientras se cumpla una condición. Podría no ejecutarse ninguna vez si no se cumple la condición.

```
#include <iostream>
using namespace std;
int main ()
{
    int n;
    cout << "Introduce el valor de partida:";
    cin >> n;
    while (n>0) {
        cout << n << ", ";
        - - n;
    }
    cout << "Terminado!" << endl;
    return 0;
}
```

RESULTADO:

Introduce el valor de partida: 8
8, 7, 6, 5, 4, 3, 2, 1, Terminado!

(b) **Instrucción do-while** : el bloque de sentencias se ejecuta al menos una vez, ya que la condición se comprueba al final del bucle.

```
#include <iostream>
using namespace std;
int main ()
{
    int n;
    do {
        cout << "Introduce un número= ";
        cin >> n;
        cout << "Tu has introducido= " << n << "\n";
    } while (n != 0);
    cout << "Hasta luego..." << endl;
    return 0;
}
```

Resultado:

```
Introduce un número= 12345
Tu has introducido= 12345
Introduce un número= 0
Tu has introducido= 0
Hasta luego...
```

(c) Instrucción **for** :

for (expresion 1; expresion 2; expresion 3) { bloque de sentencias }

→ Se repite una sentencia o bloque de sentencias un número determinado de veces mientras se cumpla la condición dada por la expresión 2. Cuando la expresión 2 no se cumple se sale del bucle.

```
#include <iostream>
using namespace std;
int main ()
{
for (int n=1; n<=10; n++) {
    cout << n << ", ";
}
    cout << "Terminado!" << endl;
    return 0;
}
```

Resultado:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, Terminado!

5.- Entrada-salida básica con ficheros.

→ Hasta el momento todas las operaciones de E/S se han realizado utilizando los flujos predefinidos **cin** (entrada estándar desde teclado) y **cout** (salida estándar hacia la pantalla), y los operadores de inserción << y extracción >>.

→ Se pueden también leer y escribir ficheros almacenados en el disco duro o en un pendrive.

→ Para realizar operaciones de E/S con archivos se debe incluir en el programa la librería fstream.

#include <fstream>

→ En C++, antes de abrir un archivo hay que enlazarlo a un flujo (stream).

(1) → Creamos un flujo y declaramos su tipo.

Hay tres tipos de flujos: entrada, salida y entrada/salida.

- **Entrada**: hay que declarar el flujo como de clase ifstream

`ifstream fentrada; // se declara el flujo "fentrada" como de clase ifstream`

- **Sálida**: hay que declarar el flujo como de clase ofstream

`ofstream fsalida; // se declara el flujo "fsalida" como de clase ofstream`

- **Entrada/salida**: hay que declarar el flujo como de clase fstream

`fstream fensal; // se declara el flujo "fensal" como de clase fstream`

(2) → Una vez creado el flujo, hay que asociarlo a un archivo usando la función

`open( )` → `open(nombrearchivo)`

`ofstream fsalida;
fsalida.open ("datos.dat");`

`ifstream fentrada;
fentrada.open ("datos.dat");`

- nombre del archivo: datos.dat

→ En la práctica, **la forma más común** de abrir un archivo es con una única sentencia que nos permite crear un flujo declarando su clase, y asociar dicho flujo con un archivo en un sólo paso:

```
ofstream fsalida("datos.dat"); // abre el archivo "datos.dat" para salida  
ifstream fentrada("datos.dat"); // abre el archivo "datos.dat" para entrada
```

→ Una vez que el archivo está abierto con un flujo asociado al mismo, podemos tener acceso a dicho archivo utilizando los operadores de inserción << y extracción >> de forma similar que con los flujos estándar cin y cout.

```
fsalida << a << " " << b << endl; // graba a y b en el fichero "datos.dat"  
fentrada >> a >> b; // lee a y b del fichero "datos.dat"
```

→ Una vez realizadas todas las operaciones de lectura o escritura de datos es necesario cerrar el archivo utilizando la función **close()** para cerrar el flujo asociado al archivo.

```
fsalida.close();
```

```
fentrada.close();
```

// ejemplo básico de lectura y escritura en ficheros

```
#include<iostream>
```

```
#include<fstream>
```

```
using namespace std;
```

```
int main(){
```

```
    int dato;
```

```
    ifstream nombre1;
```

```
    nombre1.open("nombrefichero1");
```

```
    nombre1 >> dato;
```

```
    ofstream nombre2;
```

```
    nombre2.open("nombrefichero2");
```

```
    nombre2 << dato;
```

```
    nombre1.close();
```

```
    nombre2.close();
```

```
    return 0;
```

```
}
```

Es necesario incluir esta directiva.

Se crea una flujo de entrada con el nombre "nombre1".

Se asigna un fichero concreto a dicho flujo de entrada.

Se lee el dato que contiene el fichero.
El flujo "nombre1" hace el mismo papel que "cin" en la entrada estándar.

Similar para la salida. El flujo nombre2" hace el mismo papel que cout.

Es necesario cerrar los ficheros.

→ Si no sabemos cuántos datos tiene un fichero de entrada, se puede detectar cuando se ha alcanzado el final del archivo usando la función **eof()**, ya que esta función, aplicada a un flujo, devuelve un valor distinto de cero ("verdadero") tan sólo cuando se ha alcanzado el final del archivo.

```
.....  
ifstream fentrada;  
ofstream fsalida;  
fentrada.open ("datos.txt");  
fsalida.open ("resultados.txt");  
while (!fentrada.eof()) {  
    fentrada >> numero;  
    fsalida << numero*numero << endl;  
}  
.....
```

6.- Arrays.

→ Un “array” es una serie de elementos del mismo tipo situados en posiciones de memoria consecutivas.

→ Por ejemplo, un array que contiene 5 valores enteros de tipo *int* llamado “a” se representaría de la siguiente forma:

	0	1	2	3	4
a					

En C++ **los índices comienzan en 0**.
Por tanto, los índices de un array de 5 elementos van del 0 al 4.

→ Se puede declarar un array como el de arriba así: **int a[5];**

→ Una vez declarado se puede dar valores a cada uno de sus elementos:

a[0]=8; a[1]=11; a[2]=7; a[3]=2; a[4]=-1;

a={8,11,7,2,-1};

→ También se pueden inicializar sus valores en el momento de declararlo:

int a[5] = {1,2,3,4,5};

int a[] = {1,2,3,4,5};

```
// ejemplo de arrays  
#include <iostream>  
  
using namespace std;  
  
int a[5]={1,2,3,4,5};  
int result=0;  
int main ()  
{  
    for (int n=0; n<5; n++)  
    {  
        result +=a[n];  
    }  
    cout << result;  
    return 0;  
}
```

Resultado: 15

→ **Arrays multidimensionales.**

→ Un array se puede asociar con un vector en matemáticas. En muchas ocasiones debemos resolver problemas en los que hay implicadas matrices. Para ello se pueden definir arrays bidimensionales. Por ejemplo, una matriz de 3x2 elementos se declararía así:

```
float a[3][2];
```

→ Una vez declarado se le puede dar valores a cada uno de sus elementos:

```
a[0][0]=8; a[0][1]=11; a[1][0]=7; ...
```

→ También se pueden inicializar sus valores en el momento de declararlo:

```
float a[3][2] = {8,11,7,2,-1,4}; // los elementos se introducen por filas.
```

```
float a[3][2] = {{8,11},{7,2},{-1,4}};
```

→ El número de dimensiones no está limitado a 2, podemos crear arrays de más dimensiones, es decir, con más índices:

```
double tensor[3][2][4];
```

→ Ejemplo:

```
//Representacion matricial
#include <iostream>
using namespace std;
int main() {
int n=3, m=3, p=2;
int A[3][3]={2,3,8,2,2,4,2,2,6}; // Se introducen por filas.
int B[3][2]={{4,6},{3,9},{1,8}};
cout << "Matriz A (3x3)" << endl;
for (int i=0; i<n; i++){
    cout << "fila " << i << "= ";
    for (int j=0; j<m; j++) { cout << A[i][j] << " ";}
    cout << endl; }
cout << endl << endl;
cout << "Matriz B (3x2)" << endl;
for (int i=0; i<m; i++){
    cout << "fila " << i << "= ";
    for (int j=0; j<p; j++) { cout << B[i][j] << " ";}
    cout << endl;}
return 0;
}
```

→ Punteros.

→ La memoria de un ordenador está estructurada en **bytes** o celdas elementales, cada una de ellas con una **dirección** característica. A través de la dirección de memoria de una celda determinada se podría acceder a su contenido y/o modificarlo.

→ El lenguaje C++ permite acceder directamente a las posiciones de memoria del ordenador donde están almacenados los datos utilizados, y lo hace mediante punteros.

→ Una **puntero** es un tipo de variable usada para almacenar la dirección de memoria donde se encuentra el dato al cual hace referencia. Un puntero **apunta a un dato**.

→ La sintaxis de la **declaración de una variable de tipo puntero** es: **tipo *nombre;**

int *puntero1; puntero1 es un puntero a un valor entero

double *puntero2; puntero2 es un puntero a un valor real (double)

Operador & (dirección de) :

Se aplica a una variable. Permite obtener la dirección de memoria de la variable a la que se aplica.

```
// Ejemplo operador dirección_de
#include <iostream>
using namespace std;
int main()
{
    int a=7;
    cout << "dirección de memoria=" << &a << ", a=" << a << endl;
    return 0;
}
```

Salida:
dirección de memoria=0x22ff74, a=7

→ Cuando declaramos el **array**: double A[5];

A, es un **puntero** que apunta al primer elemento de un array de 5 datos de tipo double.

A = &(A[0]); A+1 = &(A[1]); A+2 = &(A[2]); A+3 = &(A[3]); A+4 = &(A[4]);

Operador * (indirección ó valor apuntado por) :

Se aplica a una dirección de memoria o puntero. Permite acceder al valor almacenado en una variable a través de su dirección de memoria.

→ Por ejemplo:

.....

```
int *px // px se declara como un puntero a un valor entero.
```

```
int x=7; px=&x; // en el puntero px se almacena la dirección del entero x
```

```
int y;
```

```
y=*px; // en y se almacena el valor localizado en la dirección almacenada en px
```

```
cout << "dirección de memoria=" << px << ", y=" << y << endl;
```

.....

Salida:

dirección de memoria:0x22ff70, y=7

Ejemplo:

```
//punteros1
#include <iostream>
using namespace std;
int main ( )
{
    int primervalor=5, segundovalor=7;
    int *mipuntero; // variable tipo puntero a un valor entero
    mipuntero=&primervalor; //el puntero apunta a la variable primervalor
    *mipuntero=10; // ahora primervalor=10
    cout<<"primervalor es " << primervalor << endl;
    mipuntero=&segundovalor; //el puntero apunta a la variable segundovalor
    *mipuntero=20; // ahora segundovalor=20
    cout << "segundovalor es " << segundovalor << endl;
    cout << endl << "Programa terminado" << endl;
    return 0;
}
```

Sálida:
primervalor es 10
segundovalor es 20

7.- Funciones.

→ El lenguaje C++ se basa en el uso de funciones. No es posible escribir ninguna línea de código ejecutable que no pertenezca a una función. Fuera de estas sólo se pueden escribir declaraciones y definiciones.

→ Una función es un grupo de sentencias que se ejecutan cuando son llamadas desde algún punto del programa. Siempre debe existir la función **main**.

→ No se puede definir una función dentro de otra función.

→ La sintaxis de una función en C++ es:

```
tipo_retorno nombre_funcion (tipo_par1 par1, ..., tipo_parN parN){  
    declaración de variables e instrucciones;  
}
```

→ Los parámetros par1,... parN son los parámetros de entrada y se conocen como **parámetros formales**.

→ Las variables declaradas en el cuerpo de la función son **locales** y por tanto sólo son accesibles dentro del mismo. Tienen un alcance (scope) local a dicha función.

→ **Llamada a una función.**

→ Para ejecutar una función es necesario realizar una llamada a dicha función desde el código de otra función (salvo la función *main*, que no necesita ser llamada).

→ La sintaxis es:

`nombre_funcion (expresion_par1, ..., expresion_parN);`

→ Los valores `expresion_par1, ..., expresion_parN` utilizados en la llamada a la función se conocen como **parámetros reales**.

→ El número y tipo de los parámetros reales tiene que coincidir con los correspondientes de los parámetros formales.

→ Durante la llamada los parámetros reales se copian en las variables que actúan como parámetros formales, y estos se utilizan para los cálculos dentro de la función.

→ El resultado final se devuelve mediante la sentencia **return**, devolviendo el control del programa al punto donde se realizó la llamada.

→ **Ejemplo:**

```
#include <iostream>

using namespace std;

int suma (int a, int b) {
    int r;
    r = a+b;
    return (r); }

int main ( ) {
    int z;
    z=suma(5,3);
    cout << "Resultado= " << z;
    return 0;
}
```

→ En el siguiente ejemplo vemos que se pueden hacer llamadas a la función de distintas formas:

```
#include <iostream>
using namespace std;
int resta (int a, int b){
    int r;
    r = a-b;
    return (r); }

int main ( ) {
    int x=5, y=3, z;
    z = resta (7,2);
    cout << "primer resultado= " << z << endl;
    cout << "segundo resultado= " << resta (7,2) << endl;
    cout << "tercer resultado= " << resta (x,y) << endl;
    return 0;
}
```

Salida:

```
primer resultado= 5
segundo resultado= 5
tercer resultado= 2
```

→ **Funciones sin tipo: el uso de la instrucción “void”.**

→ Cuando una función devuelve un valor, este es devuelto al punto donde se realizó la llamada a través de la sentencia **return**.

→ En el caso de que la función no retorne un valor se usa la palabra reservada **void** y la sentencia **return** se puede omitir.

```
#include <iostream>
using namespace std;

void funcionerror()
{
    cout << "He terminado el contador" << endl;
    // no lleva return.
}

int main ( ) {
    int i, n=10;
    for (i=0; i<n; i++) {
        if (i==n-1)
            funcionerror();
    }
    return 0;
}
```

→ **Argumentos de una función pasados por valor.**

→ Cuando se llama a una función, todos los argumentos, excepto las matrices, son pasados por defecto *por valor*.

→ A la función se le pasa una copia del valor del argumento. Los cambios que tengan lugar en dicho valor dentro de la función (argumento formal), no modifican las variables de donde proceden los valores pasados (argumento real).

```
// Ejemplo: argumentos pasados por valor
#include <iostream>
using namespace std;
int altera (int x, int y){
    int temp;
    temp=x; x=y; y=temp;
    cout << "x=" << x << " y=" << y << endl;
}
int main ( ) {
    int x=5,y=3;
    altera(x,y);
    cout << "x=" << x << " y=" << y << endl;
    return 0;
}
```

```
Salida:
x=3 y=5
x=5 y=3
```

→ **Argumentos de una función pasados por referencia (estilo C++).**

→ En este caso no se pasa una copia del valor del argumento real a la función, sino su dirección de memoria. Todo cambio realizado en el argumento formal dentro de la función modifica también el argumento real fuera de la misma.

→ En este caso al definir los tipos de las variables en la función se declaran como referencias utilizando el símbolo **&** como prefijo. Esto le dice al compilador que el parámetro formal a utilizar es, a todos los efectos, el mismo que el parámetro real.

```
#include <iostream>
using namespace std;
void incrementa(int &a) { // referencia al parámetro a
    a = a + 1; }
int main() {
    int var = 1;
    cout << "Valor de var antes: " << var << endl; //Imprime 1
    incrementa(var);
    cout << "Valor de var después: " << var << endl; //Imprime 2
    return 0;
}
```

→ **Arrays como parámetros de una función.**

→ Arrays unidimensionales: al declarar la función especificamos en sus argumentos un identificador y un par de corchetes vacíos [] o un puntero a dicho identificador.

```
#include <iostream>
using namespace std;
```

```
void print_vector (int x[ ], int dim) {
    for (int n=0; n<dim; n++)
        {cout <<x[n] << " ";}
}
```

```
int main ( ) {
    int x[ ]= {5, 10, 15};
    int y[ ]= {2,4,6,8,10};
    print_vector (x, 3); // llamada
    cout << endl;
    print_vector (y, 5); // llamada
    return 0;
}
```

ó

```
void print_vector (int *x, int dim)
```

The diagram illustrates that in C++, an array name used as a function parameter is treated as a pointer to the first element of the array. A red arrow points from the 'x' in the array notation 'int x[]' in the first function signature to the 'x' in the pointer notation 'int *x' in the second function signature, with the letter 'ó' (or) placed above the arrow.

→ En la llamada se pasa el nombre del array sin corchetes, es decir la dirección del primer elemento de dicho array y por tanto dicho array **se pasa por referencia**.

→ Arrays bidimensionales: en la declaración de la función hay que especificar sus dimensiones que deben ser constantes o bien utilizar memoria dinámica declarando el argumento como un puntero doble (**A).

```
#include <iostream>
using namespace std;
const int fa=3, ca=3;

void muestramatriz(double matriz[fa][ca])
{
    cout << "Matriz:" << endl;
    for (int i=0; i<fa; i++) {
        for (int j=0; j<ca; j++) {cout << matriz[i][j] << " ";}
        cout << endl;
    }
}

int main()
{
    double A[fa][ca]={2,3,8,2,2,4,2,2,6};
    muestramatriz(A);
    return 0;
}
```

→ **Prototipo de una función.**

→ Hasta el momento hemos definido todas las funciones antes de que se realice la primera llamada desde el programa principal ya que el compilador necesita saber el tipo de valor que devuelve la función y el número y tipo de sus argumentos.

→ Hay una alternativa para escribir el código fuente de las funciones después del programa principal mediante la declaración de un **prototipo** de la función antes de ser usada.

→ En un prototipo se indica al compilador el nombre de la función, el tipo de valor que devuelve y el número y tipo de sus argumentos, finalizando con un punto y coma.

→ Se coloca en el bloque de las directivas de compilación, antes del "main".

→ Sintaxis:

`tipo_retorno nombre_funcion (tipo_par1 par1, ..., tipo_parN parN);`

→ Esta declaración es más corta que la definición completa, pero suficiente para que el compilador conozca el tipo de dato devuelto por la función, y el número y tipo de sus argumentos.

→ Ejemplo de llamada de funciones con prototipos.

```
#include <iostream>
using namespace std;
double potencia(double val, int potencia); // prototipo
int main() {
    double a, c;
    int b;
    cout << "Este programa calcula una potencia, a^b" << endl;
    cout << "Introduce el valor de a: ";
    cin >> a;
    cout << "Introduce el valor de b: ";
    cin >> b;
    c=potencia(a,b);
    cout << "El resultado es: " << c << endl;
    return 0;
}

double potencia(double val, int potencia){
    double resultado=1.0;
    for (int i=1; i<=potencia; i++){
        resultado *= val; }
    return resultado;
}
```

→ Punteros a funciones.

Se usan para pasar una función como argumento a otra función.

```
// Ejemplo punteros a funciones
#include <iostream>
using namespace std;
int suma (int a, int b)
    { return (a+b); }
int resta (int a, int b)
    { return (a-b); }
int operacion (int x, int y, int (*funcion)(int,int)){
    int g;
    g = funcion(x,y);
    return g; }
int main (){
    int m, n;
    m = operacion (7, 5, suma);
    cout << m << endl;
    n = operacion (20, m, resta);
    cout << n << endl;
    return 0;
}
```

Salida:

12

8

8.- Reserva dinámica de memoria con arrays.

→ Hay situaciones en las que no conocemos de antemano la memoria necesaria hasta el momento de la ejecución del programa, es decir las dimensiones del array.

→ En estos casos se utilizan técnicas de asignación dinámica de memoria que permiten reservar y liberar memoria durante la ejecución del programa de acuerdo a las necesidades. Para llevar a cabo la reserva dinámica de memoria en C++ se utilizan dos operadores: **new** y **delete**.

→ **Operador new:**

Se utiliza para reservar o asignar un bloque de memoria libre donde almacenar nuestro array. Este operador devuelve un puntero, es decir la dirección del bloque de memoria asignado dinámicamente para almacenar el array.

→ **Operador delete:**

Cuando se ha terminado de utilizar el array dinámico, es decir un bloque de memoria previamente asignado con el operador new, podemos estar interesados en liberar dicha memoria para que este disponible para otros usos. Para ello se utiliza el operador delete.

Arrays unidimensionales (vectores):

→ Para reservar memoria para un vector A de n elementos enteros, donde n se le pide al usuario por teclado durante la ejecución del programa.

→ Declaramos A como un puntero: → `int *A;`

El nombre de un array unidimensional, A, es un puntero que apunta a la dirección de memoria donde está almacenado el primer elemento de dicho vector.

→ Después se reserva memoria para un vector A de n elementos de tipo entero:

puntero = new tipo[dimensión] → `A = new int[n];`

→ Se pueden combinar las dos sentencias anteriores en una sola línea:

`int *A = new int[n];`

→ Para liberar la memoria previamente reservada se utiliza la siguiente sentencia:

`delete [] A;`

donde en este caso no hace falta especificar la dimensión n del vector.

...

int *A;

int n;

cout<< "Introduce el valor de n: ";

cin >> n;

A= new int[n]; // reserva dinámica de memoria

// rellenamos el vector

for (int i=0; i < n; i++){

 A[i]=(i*i+1);

 cout << A[i] << endl;

}

delete [] A; // se libera la memoria ocupada por A

...

Arrays bidimensionales (matrices):

→ C++ considera que **un array bidimensional es en realidad un array de punteros a los arrays que forman sus filas.**

→ Es necesario utilizar un **puntero doble ó puntero a puntero, (**A)**, que contiene la dirección del primer puntero del array de punteros a cada una de las filas del array bidimensional.

Puntero a puntero fila		Puntero a fila	Array bidimensional double A[5][3]		
A	→	A[0]	→	A[0][0]	A[0][1] A[0][2]
A+1	→	A[1]	→	A[1][0]	A[1][1] A[1][2]
A+2	→	A[2]	→	A[2][0]	A[2][1] A[2][2]
A+3	→	A[3]	→	A[3][0]	A[3][1] A[3][2]
A+4	→	A[4]	→	A[4][0]	A[4][1] A[4][2]

A es un puntero que apunta a un array de 5 punteros (A[0], A[1], A[2], A[3], A[4]).

A[0] es un puntero que apunta al primer elemento de un array de tres elementos (A[0][0], A[0][1], A[0][2]).

A[1] es un puntero que apunta al primer elemento de un array de tres elementos (A[1][0], A[1][1], A[1][2]).

etc.....

→ **Matriz A de tipo double de *fa* filas y *ca* columnas.**

→ Hay que declarar A como un **puntero doble**: → **double **A;**

La utilización de un puntero doble (****A**) implica que en este caso la reserva dinámica (y la posterior liberación) de memoria se lleva a cabo en dos etapas:

→ **Etapla (a)**: se reserva memoria para el array de punteros que apunta a las filas de A:

puntero = new tipo[dimension] → **A=new double* [fa];**

En este caso el tipo es **double***, es decir puntero a double, ya que estamos reservando memoria para un array de *fa* punteros que apuntan a las *fa* filas de A.

→ **Etapla (b)**: Se reserva memoria para cada uno de los *ca* elementos que forman cada una de las *fa* filas de la matriz A:

for (int i=0; i < fa; i++)
puntero = new tipo[dimension] → { A[i]=new double[ca];}

→ Observar, que dichas filas están apuntadas por los punteros A[i], y que en este caso el tipo es double, ya que estamos reservando memoria para los elementos de tipo double que forman la matriz A de *fa* filas por *ca* columnas.

→ Conviene insistir en que en el ejemplo anterior:

A es un puntero a un array de *fa* punteros a datos de tipo double.

A[i] es un puntero a un array de *ca* datos de tipo double.

→ Para liberar la memoria hay que proceder a la inversa, también en dos etapas, como se muestra a continuación:

```
for (int i=0; i < fa; i++) {  
    delete[] A[i];  
}  
delete [] A;
```

```
...  
double **A;  
int fa, ca;  
cout << "Introduce numero de filas: ";  
cin >> fa;  
cout << endl;  
cout << "Introduce numero de columnas: ";  
cin >> ca;
```

```
// Reserva dinámica de memoria.
// etapa (a). Se reserva memoria para el array de punteros que apuntan a las filas de A
A=new double* [fa]; //etapa (a)
// etapa (b)
//Se reserva memoria para cada uno de los ca elementos
// que forman cada una de las fa filas de la matriz A.
for (int i=0; i < fa; i++)
    { A[i]=new double[ca];} // etapa (b)

// rellenamos la matriz
for (int i=0; i < fa; i++){
    for (int j=0; j < ca; j++){
        A[i][j]=2*i+3*j;
    }
}
// se libera la memoria ocupada por A en dos etapas.
for (int i=0; i < fa; i++) {
    delete[] A[i];}
delete [] A;

...
```

→ **Direcciones electrónicas de interés:**

<https://sourceforge.net/projects/orwelldvcpp/files/Setup%20Releases/>

<http://www.cplusplus.com/>

<https://www.embarcadero.com/es/free-tools/dev-cpp>

→ **Libros de programación:**

- Programación en C++. Serie Schaum. Luis Joyanes Aguilar.
- C++ Programming Language. Bjarne Stroustrup. Addison Wesley.
- Programming with C++. John Hubbard. Schaum.

→ **Libros de consulta de la asignatura:**

- Análisis Numérico con Aplicaciones. C.F. Gerald y P.O. Wheatley. Editorial Addison-Wesley.
- Análisis Numérico. R.L. Burden, J. Douglas Faires. Editorial International Thomson Editores.
- Cálculo Numérico. B. Carnahan, H.A. Luther y J.O. Wilkes. Editorial Rueda.
- Elementary Numerical Analysis. Kendall Atkinson. John Wiley and Sons.

Apéndice: errores numéricos: definición, fuentes, ejemplos.

→ Cuando se trabaja con ordenadores se maneja una cantidad finita de cifras, lo que provoca cierta imprecisión:

$$\sqrt{3} \cdot \sqrt{3} = 3 \quad (\text{en cálculo analítico})$$
$$\text{pero } \sqrt{3} \cdot \sqrt{3} \neq 3 ; \quad \sqrt{3} \cdot \sqrt{3} \approx 3 \quad (\text{en cálculo numérico})$$

→ Esto se conoce como **errores de redondeo** y surgen por aproximar números mediante notación de coma flotante.

→ El conjunto de números que maneja un ordenador es finito (conjunto discreto). Entre un nº determinado y el siguiente existe un hueco. Todos los números posibles en ese intervalo o hueco se deben aproximar por uno de los dos valores de los extremos de dicho intervalo.

→ Por ejemplo, en tipo “float” (coma flotante, 4 bytes) el nº 1/2 en binario es:

0 01111111 100000000000000000000000

está situado entre el inmediato anterior y el posterior:

0 01111111 011111111111111111111111 (0.49999988079071)

0 01111111 100000000000000000000001 (0.500000119209289)

→ La aproximación a uno de los valores posibles puede hacerse por **redondeo** (se aproxima al número más próximo) o por **truncamiento** (se aproxima al menor de los dos números).

→ Otro tipo de error frecuente es el **error de propagación**. Muchos de los métodos del cálculo numérico se basan en procesos iterativos que implican cálculos sucesivos basados en el resultado de la iteración anterior. Si dicho resultado tiene un cierto error, aunque sea pequeño, su magnitud aumentará con el número de iteraciones, pudiendo resultar en un error elevado.

→ También nos encontramos con frecuencia con **errores de truncamiento**, término que se usa para designar el error cometido al truncar una serie infinita, como en el caso de las series de Taylor.

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

→ Cuando en un cálculo aparece un número más pequeño que el menor n° posible se produce **subdesbordamiento** (***underflow***). Puede aparecer un mensaje de error y el resultado se iguala a cero.

→ Cuando se llega a un número mayor que el más grande se produce **desbordamiento** (***overflow***). Puede aparecer un mensaje de error y en ocasiones se detiene la ejecución del programa.